From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability

Anonymous Author(s)

Abstract

empty

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software creation and management*.

Keywords

APT defense evaluation, control-point attribution, stage-aware scoring, security benchmark

ACM Reference Format:

Anonymous Author(s). 2026. From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability. In *Proceedings of THE ACM WEB CONFERENCE 2026 (Conference acronym 'WWW).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Advanced Persistent Threats (APTs) have become a major threat to national, enterprise, and critical-infrastructure cybersecurity. According to M-Trends 2026, the global median dwell time rose to 14 days in 2025, while the median dwell time for cyber-espionage intrusions reached 122 days[4]. Verizon DBIR 2025 reports that credential abuse and vulnerability exploitation remain major initial attack vectors, and that the share of breaches involving third parties doubled to 30%[13]. IBM's 2024 Cost of a Data Breach report further shows that the global average cost of a data breach reached $4.88 million, and that about 70% of affected organizations experienced significant or moderate business disruption[5]. In response, organizations deploy layered defense stacks, including Endpoint Detection and Response (EDR), Next-Generation Firewalls (NGFW), Security Information and Event Management (SIEM), Privileged Access Management (PAM), and email security gateways.

However, deployment does not imply effectiveness. A defense stack with more products is not necessarily more capable, and a system that fails to stop an entire APT campaign is not necessarily useless. APT campaigns are multi-stage, long-lived, and heterogeneous: an attacker may enter through phishing, public-facing vulnerabilities, or credential abuse; then establish command and control, collect credentials, move laterally, evade detection, and eventually exfiltrate data or cause impact. These characteristics impose three requirements on defense evaluation. First, the evaluation should distinguish where in the attack chain a defense succeeds, because blocking initial access is not equivalent to detecting exfiltration after compromise. Second, it should account for alternative and dependent attack paths: in some stages the attacker only needs one successful path, while in others all required actions must succeed. Third, it should explain which defensive control failed first, because the same missed detection may be caused by weak identity controls, insufficient isolation, missing endpoint hardening, or poor observability. Therefore, APT defense evaluation should not only answer whether an attack was stopped, but also explain which stages, control domains, and product categories are responsible for defensive success or failure.

Existing evaluation and scoring approaches only partially address this problem. MITRE ATT&CK Evaluations provide detailed step-level observations of product behavior in adversary-emulation scenarios[10]. Such results are valuable for understanding whether a product observed, detected, or protected against specific attack steps, but they do not directly translate those observations into stage-aware scores or explain which victim-side control point was the first failed control. Other work evaluates system security using vulnerability severity, attack difficulty, risk values, or attack-graph-based reachability[1, 7, 15]. These methods support quantitative risk analysis, but their main concern is usually which vulnerability or path is risky, rather than how a deployed defense stack performs against action-level APT behaviors. As a result, three gaps remain: existing scores are often black-box summaries, attack stages are frequently treated without differentiated defensive value, and missed or weakly handled actions are rarely attributed to concrete defensive control failures.

These gaps motivate a control-point-oriented intermediate representation. Instead of scoring a defense result directly from raw attack steps, this paper first maps each attack action to the victim-side control point that failed first and is most worth repairing. This representation is inspired by the observation that the same ATT&CK technique may correspond to different repair actions in different contexts: credential use may originate from brute force, credential dumping, hard-coded secrets, or phishing, each requiring a different defensive response. To make this idea usable for scoring, three challenges must be addressed. The attribution schema must be stable enough to classify heterogeneous APT actions; the attack-stage dimension must be modeled without forcing the control domains into a fixed kill-chain order; and action-level observations must be aggregated according to the logical structure of each stage rather than by a simple average.

To address these challenges, this paper proposes a stage-aware APT defense capability scoring framework based on control-point attribution. The framework contains three layers. First, a control-point attribution layer classifies each attack action by the victim-side first-failed control point, using 9 top-level domains and 94

Conference acronym 'WWW, Dubai, United Arab Emirates

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

leaf-level control points. Second, a benchmark construction layer enriches each action with a canonical ATT&CK phase, a defense opportunity map, and a defense-technology crosswalk that maps control points to defense product categories. Third, a scoring engine aligns tested defense results with benchmark actions, assigns action-level hit values for PREVENTED, BLOCKED, DETECTED, and MISSED outcomes, and aggregates them into stage-level, playbook-level, phase-wise, control-domain, and product-category scores. The stage aggregation is explicitly modeled: OR stages use the weakest covered alternative, AND stages use the strongest interruption point, and Cluster stages use weighted averaging.

The main contributions of this paper are as follows:

- We propose a control-point attribution schema for APT defense evaluation, shifting the evaluation unit from 'what the attacker did" to 'which victim-side control failed first." The schema covers 9 top-level domains and 94 leaf-level control points, enabling explainable attribution of defensive weaknesses.
- We design a benchmark extension architecture that connects control-point attribution with stage-aware scoring. It includes canonical phase mapping, a defense opportunity map, and a defense-technology crosswalk, allowing defense results to be analyzed by attack phase, control domain, and product category.
- We develop a stage-aware scoring model that separates action-level hit values from stage importance and formalizes OR, AND, and Cluster stage semantics. This avoids treating all APT steps as equally valuable and supports interpretable aggregation from actions to playbooks.
- We evaluate the framework on 53 APT playbooks containing 1,758 attack actions. The results validate the mathematical behavior of the scoring engine and show that no single product category can independently cover the full APT attack surface, highlighting the need for coordinated defense-in-depth.

2 Methodology

2.1 Overview

The framework converts multi-stage APT playbooks and action-level defense observations into interpretable defense capability scores. Its purpose is not only to record whether an attack step was detected or blocked, but also to explain why the step should have been stopped: which victim-side control point first failed, which defense product categories were expected to cover that weakness, and whether the evaluated defenses actually prevented, blocked, or detected the action.

Fig. 1 illustrates the overall workflow. The process has three components: benchmark construction, defense scoring, and score reporting. During benchmark construction, raw APT playbook actions are annotated using a control-point attribution schema. Each action is mapped to the victim-side control point that first failed and is most relevant for repair. The annotated actions are then enriched with canonical attack phases, control-point-to-defense mappings, expected defense product categories, confidence labels, and stage aggregation semantics. The output of this step is a defense opportunity map, where each action is represented as a scoring-ready record.

During defense scoring, the evaluated system provides action-level observation results. Each observation is aligned with its corresponding action record in the defense opportunity map and translated into one of four hit levels: PREVENTED, BLOCKED, DETECTED, or MISSED. These levels capture different degrees of defensive success, from prevention to complete miss. The scoring engine then aggregates action-level hit values into stage-level and playbook-level scores using the predefined aggregation semantics and canonical phase weights.

The final output is not a single score. The framework reports an overall defense capability score together with phase scores, control-point domain scores, product category scores, and action level attribution reports. As a result, the score can be traced back to specific playbooks, stages, actions, and failed control points, which directly supports stage aware and attribution driven APT defense evaluation.

2.2 Defensive Weakness Attribution

We first map each attack action to the victim-side control point that first failed and is most relevant for repair. A control point refers to a defensive mechanism, configuration, policy, or operational process that can be evaluated and remediated independently. In this sense, a successful attack action indicates that a victim-side control was absent, misconfigured, bypassed, or insufficiently enforced.

This scope excludes actions that do not correspond to a directly repairable control point on the victim side. Pre-compromise reconnaissance, attacker-side infrastructure preparation, and similar external prerequisite activities are retained for playbook completeness, but they are not included in primary weakness statistics or attribution-based scoring unless a concrete victim-side control failure can be identified.

This attribution step is necessary because an ATT&CK label tells us what the attacker did, but it does not tell us which defense failed. The same ATT&CK technique may be enabled by different weaknesses in different environments. For example, a credential-related action may result from weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, or successful phishing. These cases may share similar attack labels, but they imply different repair actions. Therefore, we interpret each action from the defender's perspective: given this action, which control point should be treated as the primary failure to repair?

Following this view, we use a structured control-point schema to organize recurring victim-side failures. The schema is not organized as an attack sequence. Instead, it separates defensive failures along four design dimensions: security function, architectural location, defensive lifecycle phase, and trust source. Security function distinguishes failures in identity, authorization, execution control, communication protection, and data protection. Architectural location separates externally exposed entry points from internal control failures. Defensive lifecycle phase separates prevention-oriented controls from observability, response, and resilience controls. Trust source captures weaknesses introduced through externally supplied

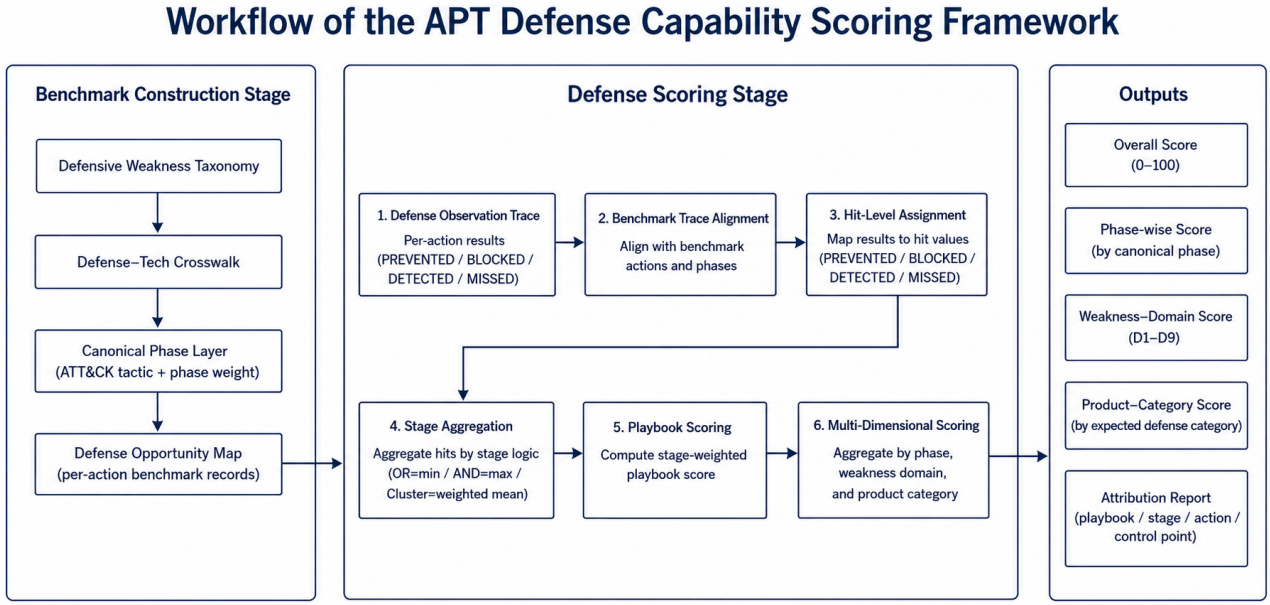

Figure 1: Workflow of the APT defense capability scoring framework.

Table 1: Top-level control-point domains.

Domain	Scope
D1	Identity, credential, and trust-root failures
D2	Authorization, isolation, and boundary-control failures
D3	Execution, host, and runtime-protection failures
D4	External exposure and remote-entry failures
D5	Communication and protocol-design failures
D6	Data protection, cryptographic-material, and recovery failures
D7	Observability, detection, and response failures
D8	External trust-input failures
D9	Availability and resilience failures

inputs, such as phishing messages, malicious documents, third-party collaboration, and supply-chain entry points.

The schema contains nine top-level control-point domains and 94 fourth-level leaves. The top-level domains provide stable categories for attribution and reporting, while the leaf nodes capture more specific repair targets. In other words, the domains support interpretable scoring, and the leaves provide the operational granularity needed for remediation.

Each in-scope action is assigned exactly one primary weakness leaf. This single-label rule keeps later scoring traceable. If a single action had multiple primary attributions, the same failure could be counted several times, and score loss could not be traced to a clear remediation target. To keep attribution consistent, the schema relies on explicit boundary rules and a primary-attribution decision principle: if only one control point could be repaired, the selected leaf should be the one whose repair would most directly prevent or disrupt the action.

Some actions still involve more than one relevant weakness. Secondary weakness tags are used to record such supporting conditions, including dual-control dependencies and entry–root-cause splits. These tags help explain attack chains, but they are not counted as primary attribution results and do not replace the primary leaf used for scoring.

In summary, defensive weakness attribution converts raw attack actions into repair-oriented control-point failures. This attributed action set then becomes the basis for the next steps of the methodology: canonical phase assignment, defense opportunity mapping, and stage-aware scoring.

2.3 Benchmark Construction

After attributing defensive weaknesses, the next step is to convert the attributed playbook actions into benchmark records that can be used for scoring. The attribution result alone is not sufficient for scoring: it tells us which control point failed, but it does not yet specify the attack stage, the expected defense product categories, or the aggregation semantics used later by the scoring engine. Benchmark construction adds these labels without changing the primary attribution result.

The benchmark construction process has three components. First, each action is mapped to a canonical attack phase using ATT&CK. Second, each control-point leaf is mapped to the defense product categories that are expected to address it. Third, the enhanced action log is written into the defense opportunity map, which will serve as direct input to the scoring phase.

2.3.1 Canonical Phase Assignment. The attribution domains describe where the victim-side defense failed, rather than when the action

occurs in the attack chain. However, scoring requires stage information because blocking an early-stage action and detecting a late-stage action should not have the same defensive value. Therefore, we add a separate canonical phase layer.

For each action, the framework assigns a canonical phase based on its ATT&CK technique. We use the ATT&CK Enterprise tactics as the standard phase vocabulary. Some ATT&CK techniques can be associated with multiple tactics, we resolve this problem using a predefined technique-to-phase mapping, the mapping was constructed during benchmark preparation according to the typical use of each technique in APT playbooks. Each technique is mapped to a single canonical phase before scoring, and this mapping remains fixed during defense scoring. Stage-level phases are then derived from the action-level phases, for example by majority voting among the actions in the same playbook stage. This phase assignment is an additional benchmark field; It does not change the control-point attribution described in the previous subsection.

The canonical phase field is later used by the scoring model to apply phase weights. The basic intuition is that earlier interruption has higher defensive value, while late-stage detection provides less benefit because the attacker has already progressed through much of the playbook.

2.3.2 Defense-Category Mapping. The primary weakness leaf identifies the failed control point, but a scoring benchmark also needs to know which type of defense product should be expected to address that failure. For this purpose, we build a defense-category mapping between control-point leaves and defense product categories.

Each fourth-level leaf is mapped to two sets of defense categories. The first set is the primary defense category set, which contains product types that should directly prevent or block the corresponding weakness if deployed and configured correctly. The second set is the compensating defense category set, which contains product types that may detect, mitigate, or partially compensate for the weakness, but are not the most direct repair target.

For example, a brute-force action attributed to weak password and anti-brute-force policy is primarily related to identity and access management, while privileged access management or log monitoring may provide compensating support. Similarly, a C2 communication action attributed to missing egress proxy enforcement may be primarily related to network access control or next-generation firewall capabilities, while SIEM or IDS may provide secondary visibility. This mapping allows later scoring to evaluate a product only against the actions that fall within its expected coverage set, rather than applying the same action set to all product categories.

2.3.3 Defense Opportunity Map. The final output of benchmark construction is the defense opportunity map. Each record in the map corresponds to one attack action and combines the information needed for scoring: the playbook identifier, stage number, action description, ATT&CK technique, primary control-point leaf, top-level domain, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

The opportunity map keeps the primary attribution unique, but it also records additional defense opportunities when they are available. In particular, secondary weakness tags can be expanded into additional leaves and defense categories. This design preserves the repair-oriented primary attribution while still allowing the benchmark to represent layered defense opportunities.

The opportunity map also records the aggregation semantics used for each playbook stage. These semantics specify how the action-level hit values within a stage should be combined during scoring. For example, a stage may represent alternative attack paths, sequentially required actions, or a loose cluster of related actions. The detailed aggregation rules are defined in the scoring model.

Table 2 summarizes the main fields used in the defense opportunity map.

Through these steps, benchmark construction turns attributed attack actions into structured defense opportunities. The resulting records connect four pieces of information that are required for scoring: what the attacker did, which victim-side control point first failed, which defense categories were expected to cover the action, and how the action should contribute to stage-level aggregation.

Table 2: Main fields in a defense opportunity record.

Field	Meaning
playbook, stage, action	Identifier of the attack action in the playbook
attack_id	ATT&CK technique or sub-technique identifier
primary_leaf	First-failed control-point leaf used for primary attribution
primary_domain	Top-level control-point domain of the primary leaf
canonical_phase	Standard attack phase used for phase-aware scoring
primary_defense	Product categories expected to directly address the weakness
compensating_defense	Product categories that may provide secondary detection or mitigation
confidence	Confidence of the primary attribution
stage_logic	Aggregation semantics used later at the stage level

2.4 Defense Result Alignment

After the benchmark records are constructed, the evaluated defense system provides action-level observation results. Each observation describes how the defense system responded to a specific attack action in the playbook. The purpose of defense result alignment is to match these observations with the corresponding records in the defense opportunity map and convert them into normalized hit values for scoring.

We represent the defense observation trace as a set of action-level records. Each record contains the playbook identifier, the stage number, the action field used in the benchmark, and the observed result. Each defense observation is matched to the action record in the defense opportunity map with the same playbook identifier, stage number, and action field. After alignment, the observed result is associated with the action's control-point attribution, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

Table 3: Defense observation results and hit values.

Result	Hit	Meaning
PREVENTED	1.0	The attack path is architecturally removed before the action can occur.
BLOCKED	0.9	The action is attempted but synchronously interrupted by the defense system.
DETECTED	0.5	The action is observed or alerted, but not stopped.
MISSED	0.0	The action is neither prevented, blocked, nor detected.

Actions without a matched defense observation are treated as missed actions. Under this rule, an action receives a positive hit value only when the evaluated defense system reports that it was prevented, blocked, or detected. Otherwise, the action is treated as MISSED. It also keeps the benchmark comparable across different defense systems: every system is evaluated against the same set of benchmark actions, and missing observations are interpreted as lack of demonstrated coverage.

Each matched observation is then converted into one of four hit levels. PREVENTED denotes architectural prevention, where the attack path is removed before the action can be attempted. BLOCKED denotes real-time interruption, where the action is attempted but stopped by the defense system. DETECTED denotes successful observation or alerting without stopping the action itself. MISSED denotes the absence of prevention, blocking, or detection. These levels are mapped to numerical hit values used by the scoring model.

The distinction between PREVENTED and BLOCKED is important. Prevention means that the attack path is unavailable by design, such as when an egress policy makes an unauthorized destination unreachable. Blocking means that the action is observed and interrupted during execution, such as when an endpoint product terminates a malicious process. Both are stronger than detection-only outcomes, but prevention is assigned a slightly higher hit value because it does not depend on runtime recognition of the specific attack instance.

This alignment step produces a unified action-level scoring input. Each benchmark action is associated with both an expected defense opportunity and an observed defense result.

2.5 Stage-Aware Scoring Model

After defense result alignment, each benchmark action has an action-level hit value and a set of benchmark metadata, including its canonical phase, attribution confidence, and stage aggregation semantics. The scoring model uses these fields to compute defense scores at three levels: action, stage, and playbook. The main design principle is to separate defense quality from stage importance. The stage score measures how well the defense performed within a stage, while the stage weight determines how much that stage contributes to the playbook-level score.

Let a denote an attack action. Its hit value $h(a) \in [0, 1]$ is obtained from the aligned defense result described in the previous subsection. The action weight $w(a)$ is defined as

$$w(a) = W_{\text{phase}}(\phi(a)) \cdot C_{\text{conf}}(\gamma(a)),$$

where $\phi(a)$ is the canonical phase assigned to action a, W_{phase} is the phase-weight function, $\gamma(a)$ is the attribution confidence label, and C_{conf} is the confidence factor. In our benchmark, the confidence factor reduces the contribution of actions whose attribution is less certain. For example, high-, medium-, and low-confidence actions can be assigned factors of 1.0, 0.8, and 0.5, respectively.

For a playbook stage s, let A_s be the set of actions in that stage. The stage score S_s is computed according to the aggregation semantics of the stage. We use three aggregation types: OR, AND, and Cluster.

For an OR stage, the attacker only needs one action in the stage to succeed in order to proceed. Therefore, the defense must block all actions in the stage. The stage score is defined as

$$S_s = \min_{a \in A_s} h(a).$$

This reflects the weakest-link constraint: if any action is missed, the stage is considered poorly defended.

For an AND stage, the attacker needs all actions in the stage to succeed in order to complete the stage objective. Therefore, blocking any one required action can disrupt the stage. The stage score is defined as

$$S_s = \max_{a \in A_s} h(a).$$

This reflects the strongest defensive interruption within the stage.

For a Cluster stage, the actions do not have a strong logical dependency. They are treated as a group of related behaviors rather than as strict alternatives or strict requirements. The stage score is computed as a weighted average:

$$S_s = \frac{\sum_{a \in A_s} h(a) w(a)}{\sum_{a \in A_s} w(a)}.$$

In this case, actions with higher phase importance or higher attribution confidence contribute more to the stage score.

The stage weight is computed separately from the stage score:

$$W_s = \sum_{a \in A_s} w(a).$$

This separation avoids mixing the quality of defense within a stage with the importance of that stage in the overall playbook. In particular, OR and AND stage scores are not weighted internally, because their semantics are determined by the attacker's path structure. The weights are used when aggregating stages into a playbook score.

For a playbook P with stages $S(P)$, the playbook-level score is

$$\text{Score}(P) = \frac{\sum_{s \in S(P)} W_s S_s}{\sum_{s \in S(P)} W_s}.$$

The overall defense capability score is then computed by aggregating the scores over all evaluated playbooks:

$$\text{Score}_{\text{overall}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{Score}(P),$$

where $\mathcal{P}$ denotes the set of playbooks in the benchmark. This formulation keeps the final score traceable: a score loss can be

traced back from the overall score to a playbook, then to a stage, and finally to the individual actions and control-point failures that caused the loss.

2.6 Multi-Dimensional Score Outputs

The scoring model does not treat the overall score as the only output. A single aggregate score can indicate whether a defense system performs well on average, but does not explain where the score is gained or lost. To support diagnosis and comparison, the framework reports scores from multiple perspectives: overall performance, attack phase, control-point domain, defense product category, and action-level attribution.

The overall score summarizes the performance of the evaluated defense system in all playbooks. It is useful as a compact indicator, but is not intended to be interpreted alone. The same overall score may result from different defense profiles. For example, one system may cover many attack actions but only detect them late, while another system may cover fewer actions but block them reliably within its expected scope. Therefore, the overall score is reported together with the decomposed outputs.

The phase wise score groups results by canonical attack phase. This view answers the question: at which attack phases does the defense system lose the most score? Since the scoring model assigns higher value to earlier interception, phase wise results help distinguish early stage blocking from late stage observation. This directly supports stage aware analysis of APT defense capability.

The control point domain score groups results by the primary victim side control point domain. This view answers a different question: which type of defensive control fails most often or contributes most to score loss? For example, a low score in an identity related domain points to weaknesses in credential and trust controls, while a low score in an observability related domain indicates insufficient detection or response. This decomposition turns an aggregate score into repair-oriented feedback.

The product category score evaluates performance within the expected coverage of each defense product category. For a product category c, let A_c denote the set of actions whose defense opportunity records include c as a primary or compensating defense category. The coverage of c is the fraction of benchmark actions that fall into this expected coverage set:

$$\text{Coverage}(c) = \frac{|A_c|}{|\mathscr{A}|},$$

where $\mathscr{A}$ is the set of benchmark actions. The hit rate of c is computed only within its coverage set:

$$\text{HitRate}(c) = \frac{\sum_{a \in A_c} \tilde{h}_c(a)}{|A_c|}.$$

Here, $\tilde{h}_c(a)$ is the hit value credited to category c. Primary defense matches receive full hit credit, while compensating defense matches may receive discounted credit because they detect or mitigate a weakness but do not directly eliminate the failed control point.

Reporting both coverage and hit rate is important. Coverage describes how much of the benchmark a product category is expected to address. Hit rate describes how well it performs within that expected scope. A product with narrow coverage but high hit rate has a different defense profile from a product with broad coverage but low hit rate, even if their aggregate scores are similar.

Finally, the framework produces an action level attribution report. For each score loss, the report retains the playbook, stage, action, ATT&CK technique, canonical phase, primary control point leaf, defense categories, and observed result. This report provides the trace from aggregate score back to concrete attack actions and failed control points. As a result, the output can support both high level comparison and detailed remediation analysis.

Table 4: Prototype validation of defensive weakness attribution.

Field	Agreement	κ	Interpretation
Scope	100.0%	1.000	Almost perfect
L1 domain	96.0%	0.953	Almost perfect
L4 leaf	86.0%	0.856	Almost perfect
Confidence	82.0%	0.679	Substantial
Mapping method	82.0%	0.687	Substantial
Secondary tag	66.0%	0.173	Slight

2.7 Prototype Method Validation

The preceding subsections define the attribution schema, benchmark construction process, result alignment procedure, scoring model, and multi-dimensional outputs. Before applying the framework to real defense-system replay experiments, we perform a set of prototype validation experiments on the benchmark and scoring infrastructure. These experiments do not evaluate a deployed defense product. Instead, they test whether the proposed methodological components are reproducible, whether the control-point–to–defense mapping is reasonable, and whether the scoring engine behaves correctly under boundary and parameter-variation cases.

First, we evaluate the reproducibility of defensive weakness attribution through a pilot inter-rater reliability study. A stratified sample of 50 ATT&CK techniques and sub-techniques was independently labeled by two annotators using the same coding guide. Agreement was measured at multiple levels, including scope decision, top-level domain, leaf-level attribution, confidence label, mapping method, and secondary weakness tag. Table 4 summarizes the results.

The result shows that the primary attribution levels are reproducible in the pilot setting. The high agreement at the L1 and L4 levels indicates that the main control-point attribution can be applied consistently. In contrast, the secondary tag has much lower agreement, so it is retained as qualitative chain-context information rather than as a primary quantitative scoring field.

Second, we validate the defense-category mapping used by the Defense–Tech Crosswalk. The external validity check compares the crosswalk categories with the dataset's native control labels. Among 1,733 comparable actions, the two sources agree on 1,297

Table 5: Prototype validation of defense-category mapping.

Validation item	Result
External agreement with native control labels	74.8% (1,297 / 1,733 actions)
Highest category agreement	DLP 96.4%, NET 93.6%, EGW 90.3%
Endpoint-category agreement	AV 73.5%, EDR 71.9%
Primary defense internal agreement	97.1% (34 / 35 leaves), $\kappa = 0.485$
Compensating defense internal agreement	45.7% (16 / 35 leaves), $\kappa = -0.094$

Table 6: Prototype scoring checks with synthetic coverage traces.

Simulation	Configuration	Score
Perfect	All actions are PREVENTED	100.00
None	All actions are MISSED	0.00
Ideal EDR	EDR primary coverage → BLOCKED	44.40
Ideal NGFW	NGFW/NET primary coverage → BLOCKED	24.93

actions, giving an agreement rate of 74.8%. We also perform an internal consistency check in which an independent annotator maps a stratified sample of 35 leaves to the 29 defense categories without seeing the existing crosswalk. Table 5 summarizes these checks.

These results suggest that the primary defense mapping is stable enough to support product-category scoring. The lower consistency for compensating defenses indicates that compensating coverage is less sharply defined than primary coverage. Therefore, compensating matches are treated as partial credit and should be interpreted more cautiously.

Third, we check the mathematical behavior of the scoring model using synthetic coverage traces. These traces are not real defense experiments; they are boundary and coverage simulations. A perfect-defense trace assigns PREVENTED to every benchmark action, and a no-defense trace assigns MISSED to every action. Two idealized product-category traces simulate systems that block all actions in their expected EDR or NGFW/NET primary coverage. Table 6 reports the resulting scores.

The perfect and none traces verify the expected upper and lower bounds of the scoring engine. The ideal EDR and ideal NGFW simulations show that a single product category cannot cover the whole benchmark even when it performs perfectly within its expected scope. This result reflects the multi-stage and multi-control nature of APT defense rather than the performance of a specific commercial product.

Finally, we test the robustness of the scoring results under reasonable parameter and aggregation variations. We vary phase weights, confidence factors, and compensating-defense discounts, and we also compare alternative aggregation choices for discovery-heavy Cluster stages and playbook-level aggregation. Table 7 summarizes the main results.

Table 7: Sensitivity and aggregation robustness checks.

Check	Result
Perfect / none under all parameter settings	100 / 0 in all configurations
EDR score range	43.41–46.67
NGFW score range	23.62–27.57
EDR/NGFW ratio range	1.57–1.94
Discovery-dominated Cluster alternative	Maximum score delta 1.07
Action-count weighted playbook aggregation	Maximum score delta 0.78

The sensitivity analysis shows that the boundary behavior of the scoring model remains stable across parameter settings, and the relative ordering between the idealized EDR and NGFW traces does not change. The aggregation robustness checks also show limited score variation under alternative but reasonable aggregation choices. These results support the use of the proposed scoring model as a stable prototype benchmark infrastructure. The later evaluation section will use real defense-system observations to test how deployed defenses perform under this methodology.

3 Evaluation

4 Discussion

5 Conclusion

References

[1] Arnau Erola, Ioannis Agrafiotis, Jason R. C. Nurse, Louise Axon, Michael Goldsmith, and Sadie Creese. 2022. A System to Calculate Cyber Value-at-Risk. *Computers & Security* 113 (2022), 102545. doi:10.1016/j.cose.2021.102545

[2] FIRST. 2019. Common Vulnerability Scoring System Version 3.1: Specification Document. https://www.first.org/cvss/v3.1/specification-document. Accessed: 2026-05-26.

[3] Marc Frigault, Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2008. Measuring Network Security Using Dynamic Bayesian Network. In *Proceedings of the 4th ACM Workshop on Quality of Protection*. ACM, 23–30. doi:10.1145/1456362.1456368

[4] Google Cloud Mandiant. 2026. M-Trends 2026: Data, Insights, and Strategies From Mandiant Frontline Investigations. https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026. Accessed: 2026-05-26.

[5] IBM Security. 2024. Cost of a Data Breach Report 2024. https://www.ibm.com/think/insights/whats-new-2024-cost-of-a-data-breach-report. Accessed: 2026-05-26.

[6] Peter E. Kaloroumakis and Michael J. Smith. 2021. Toward a Knowledge Graph of Cybersecurity Countermeasures. In *Proceedings of the 2021 Workshop on Cyber Security Experimentation and Test (CSET)*. https://d3fend.mitre.org/resources/D3FEND.pdf

[7] Peter Mell, Karen Scarfone, and Sasha Romanosky. 2006. Common Vulnerability Scoring System. *IEEE Security & Privacy* 4, 6 (2006), 85–89. doi:10.1109/MSP.2006.145

[8] MITRE. 2026. MITRE ATT&CK. https://attack.mitre.org/. Accessed: 2026-05-26.

[9] MITRE. 2026. MITRE D3FEND: A Knowledge Graph of Cybersecurity Countermeasures. https://d3fend.mitre.org/. Accessed: 2026-05-26.

[10] MITRE Engenuity. 2026. ATT&CK Evaluations: Methodology Overview. https://evals.mitre.org/methodology-overview/. Accessed: 2026-05-26.

[11] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. 2006. A Scalable Approach to Attack Graph Generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*. ACM, 336–345. doi:10.1145/1180405.1180446

[12] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. 2005. MulVAL: A Logic-based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium*. USENIX Association. https://www.usenix.org/conference/14th-usenix-security-symposium/mulval-logic-based-network-security-analyzer

[13] Verizon Business. 2025. 2025 Data Breach Investigations Report. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf. Accessed: 2026-05-26.
[14] Lingyu Wang, Tania Islam, Tao Long, Anoop Singhal, and Sushil Jajodia. 2008. An Attack Graph-Based Probabilistic Security Metric. In *Data and Applications Security XXII (Lecture Notes in Computer Science, Vol. 5094)*. Springer, 283–296. doi:10.1007/978-3-540-70567-3_22
[15] Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2007. Measuring the Overall Security of Network Configurations Using Attack Graphs. In *Data and Applications Security XXI (Lecture Notes in Computer Science, Vol. 4602)*. Springer, 98–112. doi:10.1007/978-3-540-73538-0_9

A Appendix

A.1 Related Work

This section reviews the work most closely related to the problem studied in this paper. Since our goal is not to propose a new intrusion detector or a new attack simulation platform, we focus on three lines of work that are directly connected to the limitations discussed in the introduction: attack knowledge bases and product evaluations, defender-side knowledge bases, and quantitative risk or attack-path analysis.

A.1.1 Attack Knowledge Bases and Step-Level Product Evaluation. MITRE ATT&CK has become the most widely used knowledge base for describing adversary behavior. It organizes tactics, techniques, and procedures observed in real-world intrusions and provides a common language for threat intelligence, red-team exercises, and security product evaluation [8]. In practical terms, ATT&CK answers the question: *what did the attacker do?* This is an important starting point for any APT-related evaluation, because without a shared vocabulary of attacker behavior, different reports and tests are difficult to compare.

However, the question addressed in this paper is different. Knowing that an attacker used a certain ATT&CK technique does not by itself explain why the victim environment failed to stop it. The same technique may succeed because of weak password policies, missing multi-factor authentication, exposed remote services, weak execution control, insufficient data protection, or poor monitoring. These causes lead to different remediation actions, even when the attacker behavior is described by the same ATT&CK technique. Therefore, ATT&CK provides the behavioral description of the attack, but it does not provide a defender-side failure attribution.

MITRE ATT&CK Evaluations move one step closer to defense assessment by reporting how evaluated products detect or protect against emulated attack steps [10]. This line of work answers a second question: *did the product observe, detect, or block this step?* Such step-level results are useful, especially when comparing product visibility and protection behavior under a shared emulation plan. Still, they mainly produce a visibility and protection matrix. A missed or delayed detection is reported as an outcome, but the result does not explain which victim-side control point first failed. It also does not directly distinguish whether the weakness belongs to identity control, network isolation, endpoint execution protection, data protection, or observability.

This distinction is important for our work. A step-level matrix can tell defenders where a product responded, but it does not tell them how to attribute failures to repairable control points. Our framework uses ATT&CK-style attack descriptions as input, but it adds a weakness attribution layer on the defender side.

A.1.2 Defensive Knowledge Bases and Control Capabilities. D3FEND complements ATT&CK by organizing cybersecurity countermeasure techniques from a defender-oriented perspective [6, 9]. If ATT&CK asks what the attacker did, D3FEND asks a different question: *if this defensive technique is deployed, what adversary behavior or digital artifact can it counter?* This makes D3FEND useful for reasoning about defensive capabilities and for connecting adversary behaviors to possible countermeasures.

Nevertheless, D3FEND is still not a scoring model for multi-stage APT defense. It describes defensive techniques and their relationships to artifacts and adversary behaviors, but it does not decide which control point failed first in a particular attack action. It also does not define how the result of one action should be combined with other actions in the same stage, or how stage-level results should contribute to an overall playbook score.

In our setting, this difference matters. A defense knowledge base such as D3FEND can indicate what kinds of techniques may counter an attack behavior, but a scoring framework must answer a more operational question: *which victim-side control point failed in this action, and how should this failure affect the score of the defense system?* Our control-point attribution schema and defense-technology crosswalk are designed for that purpose. They do not replace D3FEND; rather, they use a complementary viewpoint centered on control-point failure and repair priority.

A.1.3 Vulnerability Scoring and Attack-Path Analysis. A third related line of work focuses on quantifying vulnerability severity and attack-path risk. CVSS provides a standardized way to measure the severity of software vulnerabilities [2, 7]. It answers the question: *how severe is this vulnerability?* This is useful for vulnerability management, but it is not designed to evaluate how a deployed defense system performs against a multi-stage APT playbook. A vulnerability may have a high severity score, but that score does not tell us whether the defense system blocks the initial access, detects the execution step, prevents lateral movement, or only observes the attack near the exfiltration stage.

Attack-graph research extends the analysis from individual vulnerabilities to multi-step attack paths. MulVAL and related work model how vulnerabilities and configurations can be combined into reachable attack paths [11, 12]. Other studies further introduce quantitative or probabilistic security metrics over attack graphs [3, 14, 15]. These methods answer questions such as: *which attack paths are reachable?* and *how risky is a path or network configuration?* More recent cyber risk quantification work also estimates potential loss under different threat and control assumptions [1].

These approaches are valuable, but their unit of analysis is usually a vulnerability, a configuration dependency, an attack path, or a loss distribution. Our unit of analysis is an APT action mapped to a victim-side defensive weakness and then aggregated through stage and playbook logic. In other words, attack graphs mainly quantify attacker reachability or path risk, whereas our framework quantifies defense performance over a given set of APT playbooks.

A.1.4 Position of This Work. The above work provides important foundations, but each line answers a different question. ATT&CK answers what the attacker did. ATT&CK Evaluations answer whether a product observed, detected, or blocked a step. D3FEND answers what a defensive technique can counter. CVSS and attack graphs

answer how severe a vulnerability is or how risky an attack path may be.

This paper addresses a different question: *which victim-side control point first failed, and how should that failure contribute to a stage-weighted defense score?* To answer it, we build a control-point attribution schema, map APT actions to first-failed victim-side control points, connect these control points to defense product categories, and aggregate action-level results into stage-level and playbook-level scores. This design directly addresses black-box scoring by producing decomposable scores, addresses undifferentiated stage value through phase weights, and addresses missing attribution by mapping each action to a repairable victim-side control-point failure.

Received 7 October 2025; revised 24 December 2025; accepted 13 January 2026